

Week 8 Lab: Employee Management System (Dictionaries and File I/O)

Introduction

This is Phase 3 of a 4-phase course project. In this lab you modify your Phase 2 program to use dictionaries instead of parallel lists and add file storage for permanent data persistence. You will also simplify your data structure and enable your program to save and load employee records from a file.

What you'll add in Phase 3:

- Convert eight parallel lists to one list of dictionaries
- Search and filter functions to find specific employee records
- File I/O functions to save employee data to a file
- File I/O functions to load employee data from a file
- Data persistence across multiple program runs
- Date/time string conversion for file storage

Learning Objectives

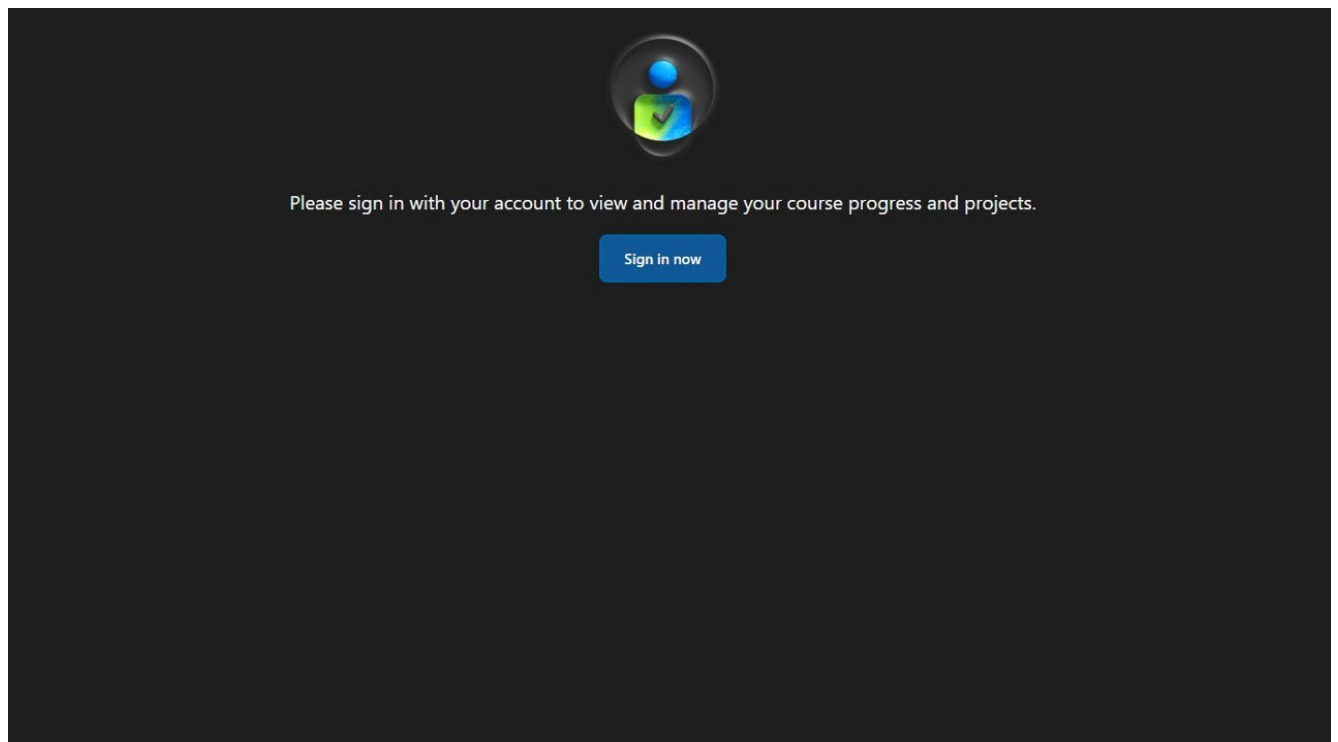
By completing this phase, you will demonstrate your ability to:

- Create and use dictionaries with multiple key-value pairs
- Convert between parallel lists and list of dictionaries
- Access dictionary values using key notation
- Use list comprehensions to extract data from dictionaries
- Create filter functions that search through data
- Write data to text files in delimited format
- Read data from text files and parse it
- Convert between datetime objects and strings
- Implement data persistence across program executions
- Handle file errors gracefully with try/except

Lab Instructions

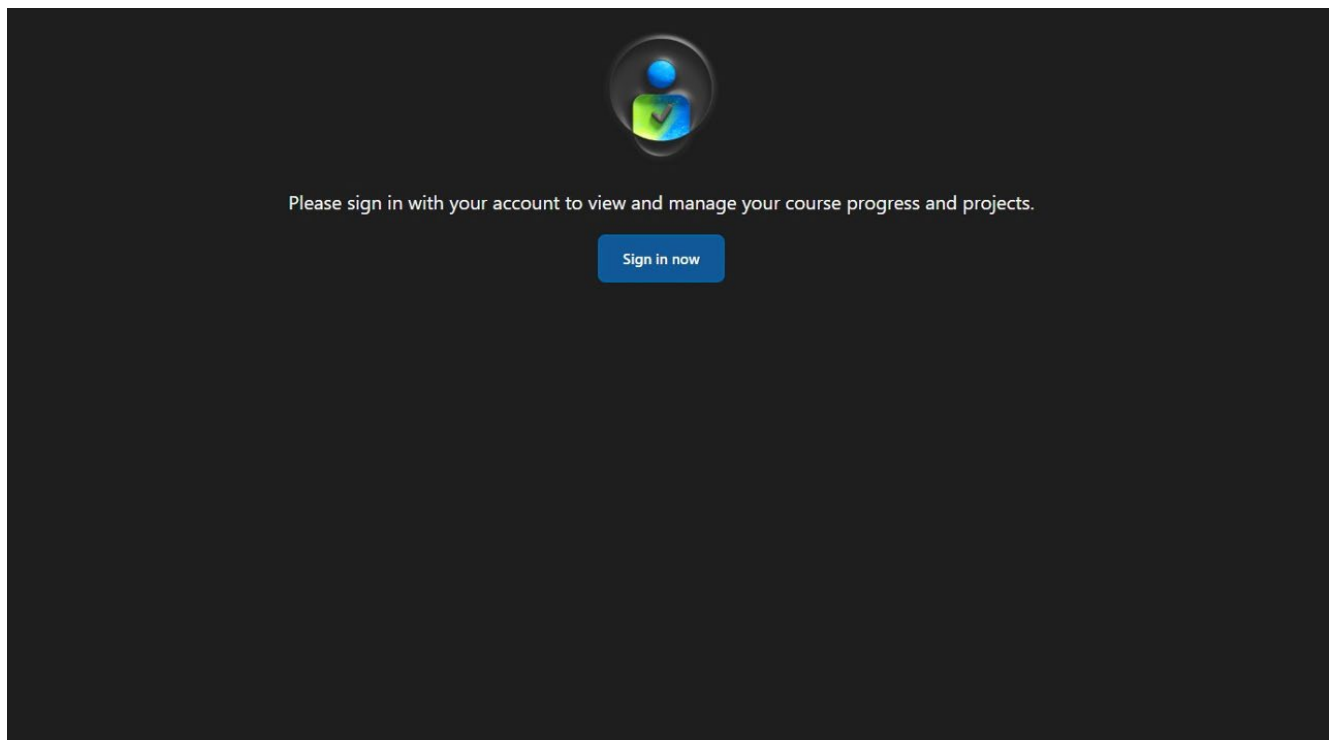
Step 1

Click on <https://vscodeedu.com/my-work/projects> to access Visual Studio Code for Education.

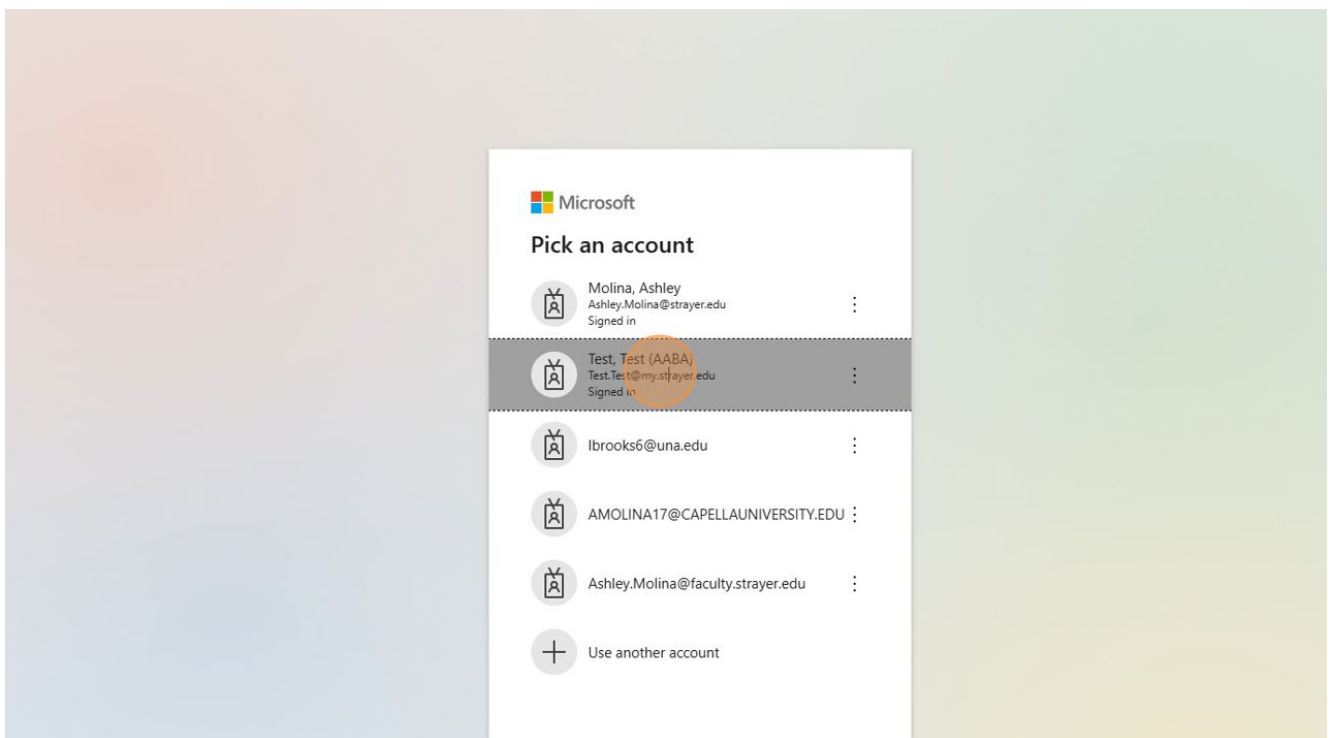
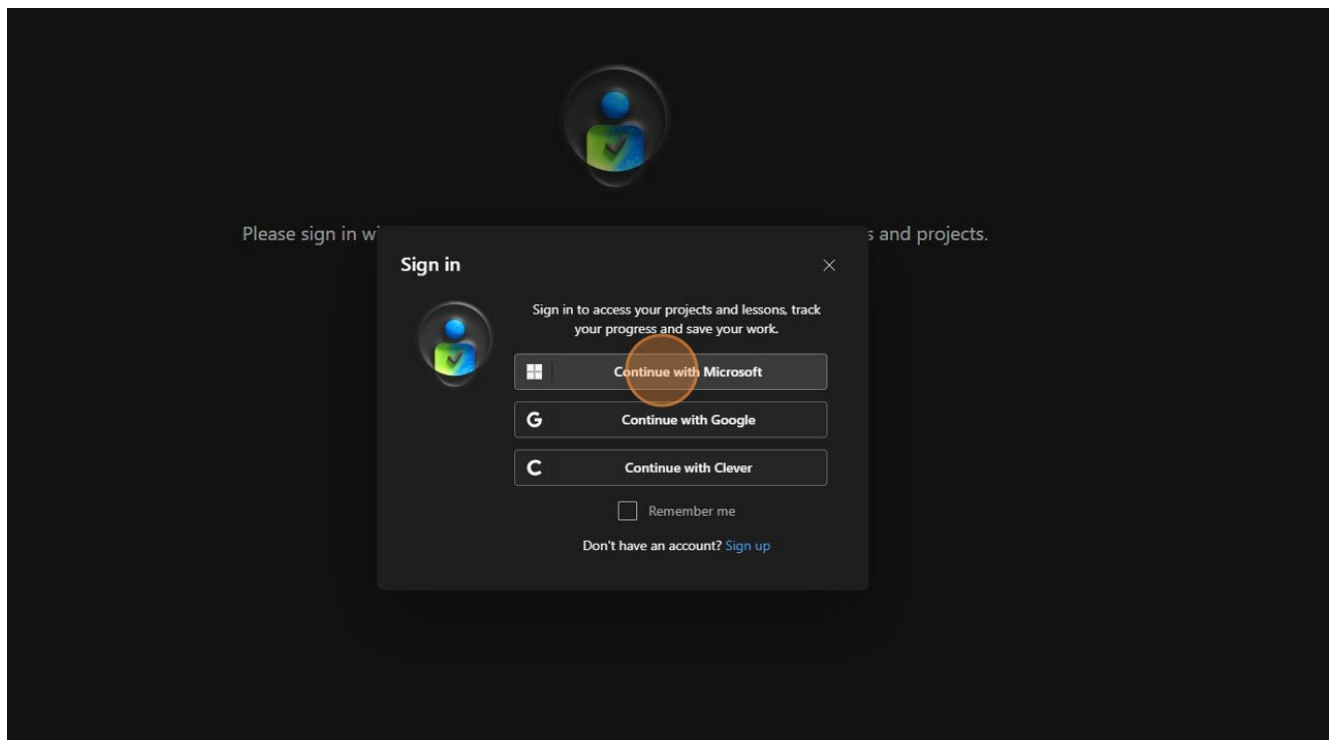


Step 2

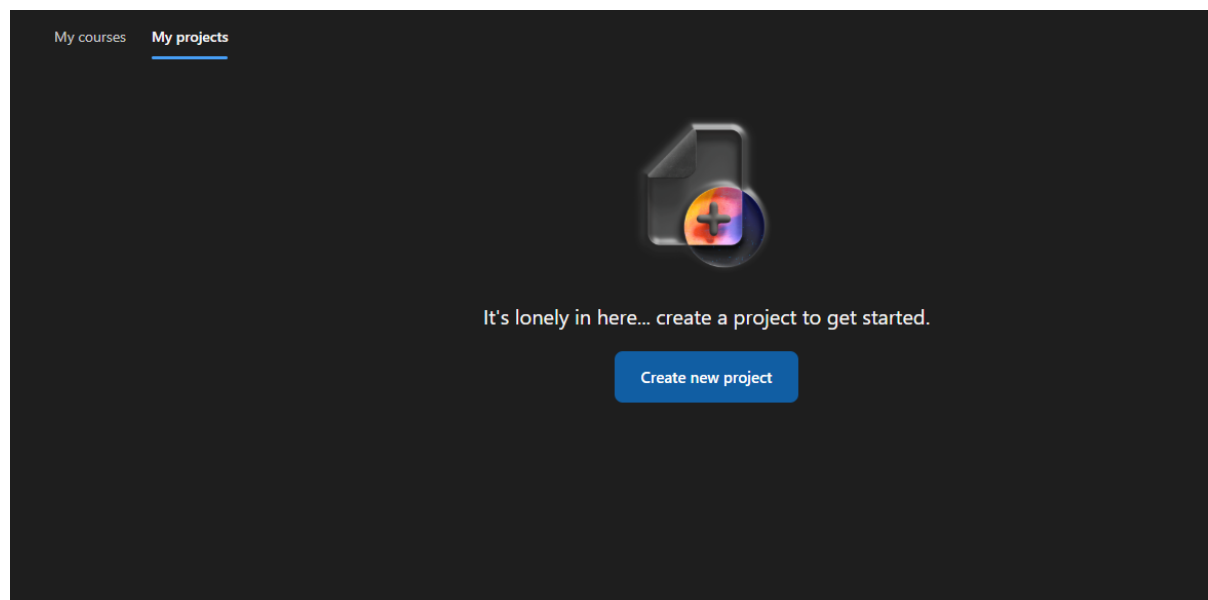
You will then Click 'Sign in now'.



Then select **'Continue with Microsoft' or 'Continue with Google'** to sign in with your already established email address. Using 'Continue with Microsoft' you will sign in with your Strayer email address that ends in @my.strayer.edu and your iCampus password. You will be prompted to log into the Strayer Systems as normal.

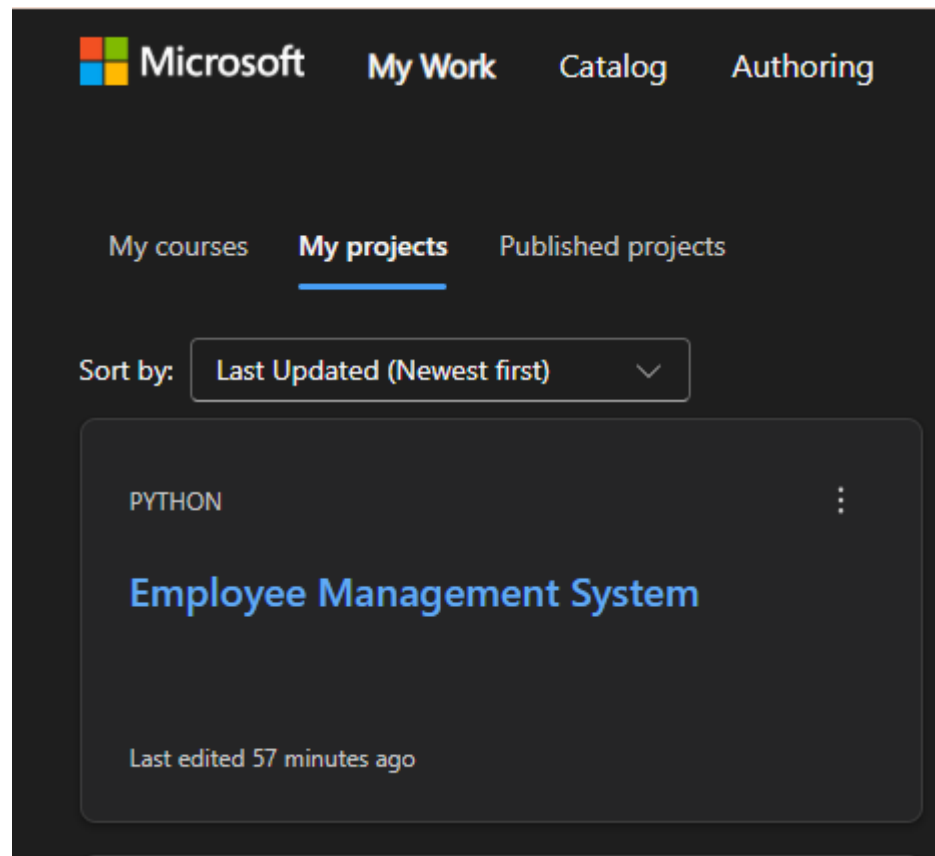


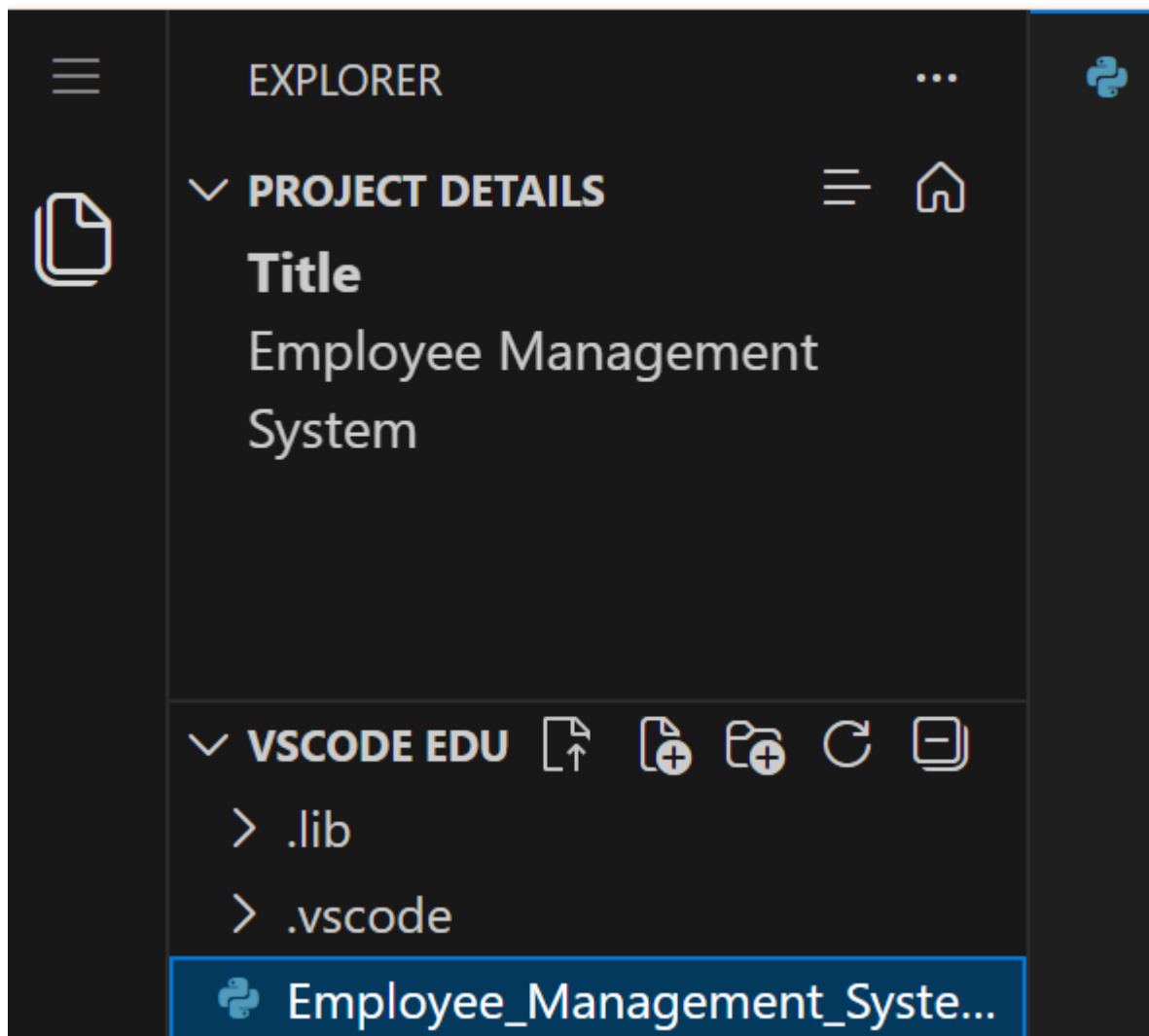
Step 3



Step 4

Open your previous project named **Employee Management System** and click on the file name `Employee_Management_System.py` from the left-hand menu once it loads to bring up your previous code.





Step 5

Edit your line 4 comment to #Phase 3.

```
Employee_Management_System.py X
1  #Your Name
2  #CIS261
3  #Employee Management System
4  #Phase 3
5
```

Step 6

Write a program to perform the following functionality.

Part 1

Convert from eight parallel lists to one list of dictionaries. In your **main()** function, replace the eight empty list initializations with ONE empty list:

- **employee_records = []**

After processing each employee (after calling **display_employee_record**), create a dictionary with nine keys instead of appending to eight lists:

- **'first_name'** – employee first name (string)
- **'last_name'** – employee last name (string)
- **'employee_id'** – employee ID (string)
- **'hours'** – hours worked (float)
- **'rate'** – hourly rate (float)
- **'gross_pay'** – gross pay amount (float)
- **'deductions'** – deductions amount (float)
- **'net_pay'** – net pay amount (float)
- **'date'** – record date (datetime object using `datetime.now()`)

Append this dictionary to the **employee_records** list.

***NOTE:** Each dictionary should contain ALL information for one employee. The list will contain multiple dictionaries (one per employee).*

Part 2

Modify your **display_all_employees()** function to work with dictionaries:

- Change to accept ONE parameter: **employee_records** (list of dictionaries)
- Use **for record in employee_records:** to iterate (instead of `range(len())`)
- Access data using dictionary key notation: **record['first_name']**, **record['gross_pay']**, etc.
- Display employee number using **enumerate(employee_records, 1)**

Part 3

Modify your **calculate_payroll_statistics()** function to work with dictionaries:

- Change to accept **employee_records** list (instead of just **gross_pays**)
- Extract gross pays using list comprehension: **gross_pays = [record['gross_pay'] for record in employee_records]**
- Continue to use `sum()`, `len()`, `max()`, `min()` on the extracted list
- Return the same four values

Part 4

Create four new search/filter functions:

find_employee_by_id(employee_records, employee_id)

- Searches for employee by ID (case-insensitive using `.lower()`)
- Returns list of matching records

find_employees_by_last_name(employee_records, last_name)

- Searches for all employees with matching last name (case-insensitive)
- Returns list of matching records

find_overtime_employees(employee_records)

- Finds all employees who worked overtime (hours > 40)
- Returns list of matching records

find_high_earners(employee_records, minimum_gross)

- Finds employees with gross pay >= minimum amount
- Returns list of matching records

Part 5

Create a **display_filtered_records()** function:

- Accept two parameters: `employee_records` (filtered list) and `filter_description` (string)
- Display header with the filter description
- Show count of matching records
- Display each record in the filtered list
- Handle empty lists with message “No matching records found”

Part 6

Create a **save_records_to_file()** function:

- Accept two parameters: `employee_records` and `filename`
- Open file in write mode using **with open(filename, 'w') as file:**
- Write each record as one line with pipe-delimited format
- Convert datetime to string using `.strftime("%Y-%m-%d")`
- Format:
 - **first_name|last_name|employee_id|hours|rate|gross_pay|deductions|net_pay|date**
- Use try/except to handle errors
- Display success message after saving

Step 7

Create a **load_records_from_file()** function:

- Accept one parameter: `filename`
- Open file in read mode using **with open(filename, 'r') as file:**
- Read each line and split by pipe: **parts = line.split('|')**
- Create a dictionary from the parsed data
- Convert date string back to datetime using `datetime.strptime(parts[8], "%Y-%m-%d")`
- Append dictionary to list
- Handle **FileNotFoundError** (file doesn't exist yet)
- Use try/except for error handling
- Return the list of employee record dictionaries

Step 8

Modify your **main()** function to implement file operations:

- Set filename variable: **filename = “employee_records.txt”**
- At the START (before the loop), load existing records: **employee_records = load_records_from_file(filename)**
- Display message showing how many records were loaded
- After the loop ENDS, save all records (old + new):
save_records_to_file(employee_records, filename)
- After displaying all employees and summary, demonstrate the search/filter functions:
 - Ask user if they want to search by employee ID
 - Ask user if they want to search by last name
 - Automatically display overtime employees
 - Automatically display high earners (gross >= \$1000)

***NOTE:** Data should accumulate across multiple runs – each time you run the program, previously saved employees should load first, then new employees are added to the list.*

Step 9

Testing Requirements

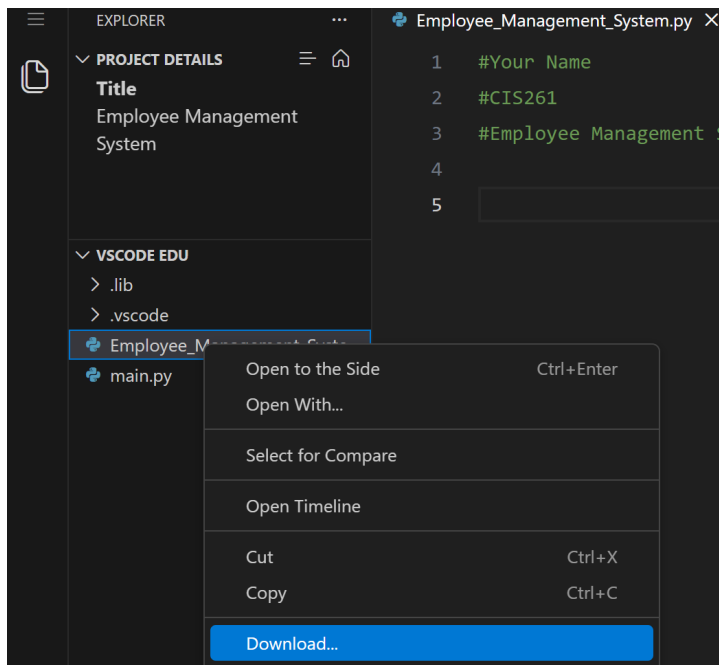
Test your program with a minimum of **3 employees** across **multiple runs** to demonstrate:

- **First run:** Enter 2 employees, verify file is created
- **Second run:** Load 2 employees from file, add 1 more, verify file now has 3 total
- All data correctly stored in dictionaries (verify by checking detailed list)
- List comprehension extracts data correctly for statistics
- Search by ID finds correct employee
- Search by last name finds correct employees
- Overtime filter shows only employees with hours > 40
- High earners filter shows only employees with gross >= \$1000
- Data persists between program executions

Saving Your Work to Your Computer

Now, we are going to download our python file for submission!

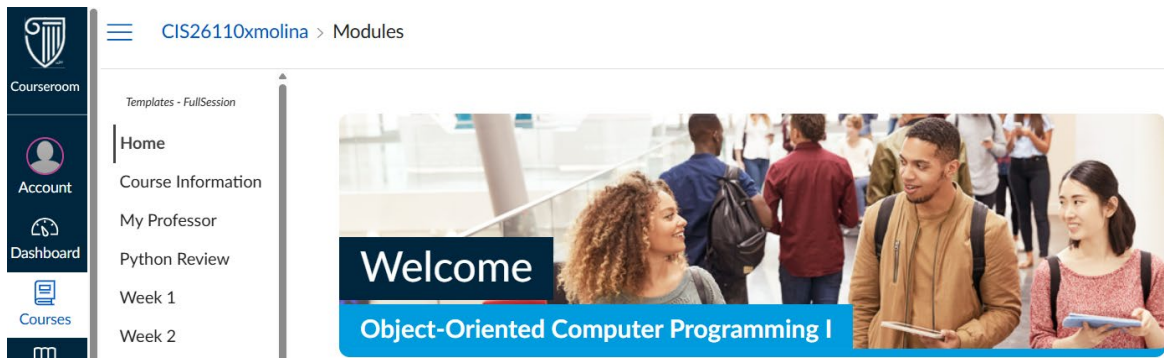
We need to download your **Employee_Management_System.py** file to your computer to prepare your submission. To do this right-click on **Employee_Management_System.py** and a pop-up box will appear. Select **Download** from the menu, to download the file to your downloads folder on your computer.



If this is the first time in here you may get a pop-up that asks ‘*vscode.dev wants to See text and images copied to the clipboard Block or Allow*’ click **Allow**. Once you ‘Allow’ you will then see your menu to download your file.

Now your python file for this project is in your computer’s downloads folder!

Go to <https://canvas.strayer.edu/> in your web browser, sign-in, and select **CIS261 Object-Oriented Programming** from your tiles.



Step 3

Select Week 8 – Lab from the Week 8 menu options

For this lab you should submit your **CIS261_Employee_Management_System.py** directly to the **Week 8 – Lab** submission area.

Step 4

Select the **Upload** option and then select **Choose a file to upload** from the submission area and select **CIS261_Employee_Management_System.py** from your selected location in your File Explorer.

When you select the file you will click **Open** to add the file to Canvas.

Once your file shows a green check mark beside it click ***“I agree to the tool’s End-User License Agreement. This assignment submission is my own, original work”*** and click **“Submit Assignment”**

Review

Well, done, you have completed the **First Step of the Employee Management System!**

Important Notes

- Must convert from 8 lists to 1 list of dictionaries
- Each dictionary must have exactly 9 keys
- Must use dictionary key notation: **record[‘key’]**
- Must use list comprehension to extract gross_pays: **[record[‘gross_pay’] for record in employee_records]**
- File format must be pipe-delimited with 9 fields per line
- Must have 8 pipes (|) per line in the file (9 fields = 8 separators)
- Date must be stored as string in format **YYYY-MM-DD**
- Must use **.strftime(“%Y-%m-%d”)** to convert datetime to string
- Must use **datetime.strptime()** to convert string back to datetime
- Must handle **FileNotFoundError** when file doesn’t exist yet
- Data must persist across multiple program runs (load at start, save at end)
 - Search functions must use **.lower()** for case-insensitive comparison

END OF DOCUMENT